Proof of Stake on Sequentia

How blocks are produced, who is allowed to produce them, and how producers are paid. This document describes the consensus mechanism as it is specified and implemented: what a stake is, how a producer is elected, how a block is certified, how tokens can stake while remaining non-transferable, how staking rights can be lent to a pool without surrendering custody, and how a pool's rewards are distributed. It explains what makes a block final, why Bitcoin's proof of work has the last word over Sequentia's own consensus, and closes with an honest account of the trust model and its limits. Written for a reader who understands Bitcoin.

1. The three commitments that shape everything else

Sequentia is a UTXO chain using Bitcoin Script rather than a general-purpose virtual machine. It is forked from Elements, the Bitcoin sidechain codebase, and inherits its transaction format and its issued-asset support. It replaces Elements' fixed-federation signed block with a stake-weighted leader and a certifying committee. The block *signature* machinery is inherited unchanged: a block is valid if it satisfies a challenge script. Proof of Stake only decides **which key must sign** each block, never how a signature is verified.

Three commitments frame the rest of this document. Each explains design decisions that would otherwise look arbitrary.

Bitcoin anchoring is supreme. Every Sequentia block references a Bitcoin block header, and Sequentia reorganises whenever Bitcoin reorganises. It does so in real time, and it does so even when that means discarding blocks the network had already treated as settled. Section 5 defines what "settled" means here and explains why anchoring is allowed to override it. This is not a safety net added afterwards; it is load-bearing in Proof of Stake specifically, because the per-block election seed is derived from the anchored Bitcoin hash. Bitcoin's proof of work is where Sequentia's randomness comes from. That is why no block producer can grind the election, and why none can bias the reward lottery described in section 10.

There is no inflation. The entire supply of the Sequence token is pre-mined. There is no coinbase issuance and no block subsidy. A producer's whole reward is the transaction fees contained in the block it produced. Wherever this document says "reward," it means fees. It is also why the coinbase rules are unusually strict: the coinbase is the only place in a block where value can be claimed.

The Sequence token has no privileged standing, with exactly one exception. Fees are payable in any accepted issued asset, and block proposers choose which assets they accept. No asset is a "native coin." The single exception is staking: only Sequence may be staked, and consensus checks the asset identifier explicitly. This document is about that exception.

2. What a stake is

This is the central idea, and it is not what most Proof of Stake designs would lead a reader to expect.

> **A stake is an ordinary unspent transaction output that matches a particular script pattern. It is never moved, never surrendered to a module, never handed to anyone. Producing a block does not spend it, reference it, or sign over it. The chain counts the money sitting in the output, and that is the entire mechanism.**

The staking script is stored *bare*, that is, unhashed, in the output. A validator therefore recognises a stake the moment the output is created, without waiting to see it spent:

```
<csv> OP_CHECKSEQUENCEVERIFY OP_DROP

    [ <bls_pubkey> OP_DROP <bls_proof_of_possession> OP_DROP ]   # optional committee
registration

    [ <liquid_locktime> OP_CHECKLOCKTIMEVERIFY OP_DROP ]          # optional vesting lock,
section 8

<pubkey> OP_CHECKSIG
```

An output counts as stake, contributing its full value as *weight*, if and only if three conditions hold:

1. Its value and asset are **explicit**, meaning unblinded, and the asset is Sequence. A confidential amount cannot carry verifiable weight, because no one can see how much it is. Sequentia is transparent by default and confidentiality is opt-in, so this costs nothing in the ordinary case.
2. Its `scriptPubKey` matches the template exactly. The parser is a closed template and rejects any trailing opcode.
3. Its relative timelock is at least the chain's minimum unbonding delay. Height-based and time-based encodings are converted to a wall-clock duration so both are judged on one axis.

The set of stakers and their weights lives in a registry that is **a pure function of the UTXO set**. It is rebuilt by a full scan when a node starts and mirrored incrementally on every block connect and disconnect. That purity buys reorganisation safety for nothing: there is no separate consensus state to roll back, only the UTXO set, which a node already rolls back correctly.

Two consequences follow, and both are load-bearing later.

- **Unbonding is simply the act of spending the staking output.** There is no unstaking transaction type and no queue. The `OP_CHECKSEQUENCEVERIFY` in the script enforces the delay directly, and the weight disappears when the output is spent.
- **A coin that cannot be spent can still stake.** Nothing about earning requires the ability to move. This is what makes staking-only vesting periods expressible at all.

Weight is summed *per public key* across all of that key's staking outputs, so the minimum-stake floor applies to a key's total rather than to any individual output.

Parameters

parameter	value	meaning
total supply	400,000,000 SEQ	fully pre-mined; no issuance, ever
minimum stake	40,000 SEQ	0.01% of supply. Below this a key is ignored by election, by sortition, and by the eligible-total denominator
slot interval	30 seconds	nominal block time, and the unit of the sortition time gate
unbonding delay	43,200 blocks (about 15 days)	minimum relative timelock for an output to count as stake
committee cap	250	a consensus rule, not a setting: a node cannot override it. The committee is the smaller of this cap and the number of eligible stakers. Sized to resist capture by a hostile third of all stake
quorum	126 of 250	a strict majority of the actual committee, plus one at odd sizes. A majority rather than two thirds, so a sleepy third cannot halt the chain
payout notice	2,880 blocks (about 1 day)	delay before an announced payout policy binds, section 8

3. Electing a producer, and certifying its block

Every block height has a seed, and the seed comes from Bitcoin:

```
seed = SHA256( the parent block's Bitcoin anchor hash || height )
```

Because that anchor hash is the output of Bitcoin's proof of work, no Sequentia participant chooses it, and no producer can grind for a favourable one. Every node derives the same seed independently, with no communication.

Two distinct things are decided from that seed: who may *produce* the next block, and who must *certify* it.

The producer, by private sortition

Each staker privately evaluates a verifiable random function over the seed with its own key, obtaining a value that nobody else can compute or predict. From that value, and from the staker's share of total stake, it derives a *slot*:

```
slot ~ uniform in [ 0, total_weight / own_weight )
```

A staker may publish a block only once `slot x 30` seconds have elapsed since the parent block. More

stake gives a statistically earlier slot, so a large staker leads more often, in proportion to its holdings. When it publishes, the producer includes its VRF proof in the block, and every node verifies that the proof really does correspond to that key, that seed, and therefore that slot.

The important word is *private*. Until a block appears, nobody, including the staker's peers, knows who is entitled to produce it. There is no published rota to attack, bribe, or censor in advance. If the earliest-slot staker is offline or unwilling, the chain does not stall and does not vote: the next staker's slot simply arrives thirty seconds later, and it produces instead.

The committee, by public schedule

Certification is the opposite: it must be public, because a validator has to know which signatures to expect. Every node computes the same schedule from the seed by giving each staker a *weighted ticket*:

```
ticket = SHA256(seed || pubkey), read as a 256-bit integer

value  = ticket / weight        # more weight gives a lower value, and an earlier place
```

Sorting by that value orders the stakers. The committee is the first 250 of them that have registered a BLS signing key, taken in schedule order. Registration is not a separate ceremony: a staker's BLS public key and its proof of possession are carried inside its own staking output, so the committee, like the weight, is a pure function of the UTXO set. The proof of possession is verified once, when the block creating that output connects.

Each committee member signs the block with its BLS key. Because BLS signatures over the same message aggregate, the certificate carried by the block is a single signature plus a bitfield naming which members of the ordered committee contributed to it. A block certified by two hundred and fifty signers is no larger than one certified by two, and a validator checks the whole certificate with one aggregate verification against the registered keys.

The block itself is signed by the producer with an ordinary ECDSA key, exactly as the challenge script requires. Only the certificate uses BLS.

The committee is capped at 250 members, and is the smaller of that cap and the number of eligible stakers, so a young network with sixty stakers runs a committee of sixty rather than an impossible one of 250. Quorum is a strict majority of the *actual* committee, plus one when that number is odd: 126 of 250.

Two numbers, two separate reasons, and they are easy to conflate. The quorum is a bare majority rather than the two-thirds familiar from Byzantine literature because a two-thirds quorum is unreachable whenever a third of the committee is merely asleep, and stakers on ordinary hardware are asleep more often than they are malicious. The *size* is 250 because size is what resists capture: a coalition holding a third of all stake will, by chance, occupy a majority of a 100-member committee within a couple of days, and of a 250-member committee about once in fifty years. Size buys safety against a hostile third; a majority quorum buys liveness against a sleepy third.

The overlap rule follows from the quorum. Any two quorums share at least two members, so no two

blocks at one height can both be certified unless at least two committee members put their registered key on both. That is the property the design is buying, but it is not free, and the next section explains what else is required to make it hold.

4. Agreeing on a block within a height

Sortition says who *may* produce. It does not say what happens when two stakers are entitled to produce at nearly the same moment, when the earliest is dead, or when the network is briefly cut in half. Agreement is reached by a round structure running inside each thirty-second slot.

Rounds, and backing a proposal

After the parent block's timestamp, members spend a *collection window* gathering whatever proposals have arrived, rather than signing the first one they see. When the window closes, every member backs the same candidate, because every member ranks candidates by the leader's VRF value and the lowest wins, and that value is a number all of them can verify. Members then flood their signature shares, and any node that has collected a quorum of shares assembles the certificate.

If no certificate forms, the round ends and the next round begins, at the same height, with the next-best leader. A dead or silent producer therefore costs one round, not a vote and not a stall. Certification proceeds at the speed of the fastest quorum rather than the slowest member: once any node holds a quorum of shares, the block is certified, and the slowest half of the committee contributes nothing to latency.

The window and round lengths grow with committee size, since share traffic and share verification are the only per-member costs. They are **local liveness timings, identical on every node but not consensus rules**. No block is valid or invalid because of them. A node that is slow simply signs late or misses a round; nothing it produces is rejected and nothing it accepts is wrong. This matters more than it sounds: it means a mistuned constant costs seconds of delay and can never split the chain.

The certificate travels alone

A certificate is a block hash, a bitfield naming the signers, and one aggregate signature: about three hundred bytes, and self-verifying against the registry. It is therefore flooded as its own object, independently of the block it certifies, which may be a thousand times larger.

This is a safety mechanism, not an optimisation. If a large block is lost to a partition while its tiny certificate crosses, a node on the far side that has seen the certificate knows the height is taken: it pins its finality floor to that block, fetches the body, and refuses to mint a rival there. Without the certificate as an independent object, the cut-off side could produce a competing block at a height that had quietly certified elsewhere.

The share-lock

Here is the failure that overlap alone does not prevent, and it is made of honest nodes.

Suppose block X gathers its final signature share just before a round boundary. Some node assembles X's certificate and begins flooding it, but flooding takes time, and the clock rolls over first. Every member that has not yet seen the assembled certificate now believes round one has begun, and dutifully signs round one's leader at the same height. Their earlier shares for X remain valid, so X certifies anyway. If the rival also reaches quorum, two blocks are certified at one height. Nobody cheated. The overlapping members signed both proposals honestly, one round apart, exactly as the clock told them to.

The **share-lock** closes this. A member that has signed a share for X at some height does not sign any other block at that height, and does not build above that height on a chain excluding X, until X can no longer certify. It releases only when the round in which it signed has ended, it has asked its peers whether a certificate for X exists (a three-hundred-byte round trip), and a grace round has passed with no answer. Under a partition, where a hidden certificate can outlast any number of rounds, the release is keyed on parent-chain progress instead of the clock: the same three-Bitcoin-block gap the anti-freeze valve uses.

The arithmetic is what makes it airtight. For X to be a live threat at all, a quorum of members must already have signed it, and all of them are locked. Fewer than a quorum remain free, so no rival can reach quorum while the lock holds. In steady state the lock costs nothing, and a dead leader costs one round.

The share-lock is also what removes the only *safety* coupling from the timing constants above. Without it, one unlucky race at a round boundary splits the chain. With it, every timing mistake in the system degrades into a few seconds of delay.

The anti-freeze valve

If the committee cannot reach quorum at all, a naive chain would stall forever. Sequentia permits a below-quorum block, down to a single signer, once the parent chain's anchor has advanced three Bitcoin blocks past the parent's, roughly thirty minutes. Liveness is restored by Bitcoin's clock rather than by weakening the safety threshold during normal operation. Such a block never becomes immediately final, it loses to any quorum-certified sibling, and the valve is suppressed entirely at any height where a certificate is known to exist.

Choosing between rival blocks

When a node sees two blocks at one height it orders them deterministically: first by certification, the block backed by more countersignature power winning; then, on a tie, by the leader's VRF value, the lower winning; and finally by the lower block hash. The block hash is the last key on purpose. It is globally deterministic, so two honest nodes that have seen the same two blocks always choose the same one, and a rare symmetric split heals itself without any node having to be told which chain to prefer.

Nothing in the ordering reorders a block that is already certified. Certification is never revisited on the basis of height, arrival rate, or anchor freshness, because a chain that reordered certified blocks by any of those could be overtaken by an attacker holding no Bitcoin hashpower at all.

5. Finality, checkpoints, and the order of precedence

A Proof of Work chain has no notion of a block being final; it only ever becomes expensive to undo. A Proof of Stake chain must say something more definite, because rewriting old history costs a stakeholder nothing once their stake is gone. Sequentia answers this in three layers, and the ordering between them is a consensus rule in its own right.

Immediate finality

Once a committee quorum has certified a block, nodes treat that block as final and will not reorganise away from it. This is affordable precisely because of the quorum-intersection property in section 3, together with the share-lock in section 4,: two rival blocks at the same height cannot both be certified without at least two committee members signing both, which is publicly attributable misbehaviour rather than an anonymous race.

Immediate finality protects a node that was online and watching. It does nothing for a node syncing from scratch, which is where the second layer comes in.

The long-range attack, and what a checkpoint is

Consider the keys that held a large fraction of stake a year ago. They have since unbonded and sold their tokens, so they have nothing left to lose. Nothing prevents them from going back to a block from a year ago and signing an entirely different history from that point forward. It costs them no electricity and no capital. A node that was online throughout will simply ignore that history, because it already finalised the real one. But a node syncing for the first time sees two histories, both properly signed by keys that genuinely held stake at the time, and has no intrinsic way to tell which is the one the world actually used. This is the *long-range attack*, sometimes called posterior corruption, and every Proof of Stake design must answer it somehow.

Sequentia answers it by borrowing Bitcoin's history. **A checkpoint is a reference to a Sequentia block, published inside a Bitcoin transaction.** Anyone may create one: it is a small tagged output containing the Sequentia block's hash and height. A node treats a checkpointed Sequentia block as finalised once two conditions hold: the block is already in that node's own validated chain, and the Bitcoin transaction carrying the reference is buried deep in Bitcoin, by default 2,016 Bitcoin blocks, roughly two weeks.

The crucial property is what a checkpoint *cannot* do. It never causes a node to seek, download, or adopt any chain. It only locks in history the node had already validated for itself. A forged checkpoint, or two conflicting ones, is therefore harmless: a node that has not independently validated the referenced block simply ignores the reference. Checkpoints add finality; they can never impose a chain.

To rewrite Sequentia history below a checkpoint, an attacker would have to rewrite two weeks of Bitcoin's proof of work. This is why the unbonding delay in the parameters table is what it is. Roughly fifteen days of unbonding exceeds the roughly fourteen-day checkpoint burial window, so a staker's coins are still locked at the moment a checkpoint consolidates over the blocks they signed. Nobody

can unbond, walk away, and then costlessly rewrite the history they helped build. The long-range window is closed.

A node may additionally be started with a hard-coded height-and-hash pair. This is a blunter instrument that protects a completely fresh sync before any block has been downloaded, and it is reject-only for the same reason: it can refuse a branch, never demand one.

Why Bitcoin anchoring outranks both

Every Sequentia block names a Bitcoin block as its anchor. If Bitcoin reorganises and the anchored block is orphaned, then **the Sequentia block is discarded, even if a committee certified it, and even if it was finalised.** Anchoring outranks checkpoints, which outrank immediate finality. A node that throws away a finalised block because Bitcoin abandoned its anchor is behaving correctly, not suffering a fault.

This ordering looks severe until one asks what it is for. Sequentia follows Bitcoin's reorganisations in real time so that a Sequentia transaction is never more settled than the Bitcoin history it refers to. That is exactly the property a cross-chain atomic swap or a Lightning payment needs: the two chains can never disagree about which of them saw an event first, because one of them is definitionally right. A design that let a finality rule veto a Bitcoin-driven reorganisation would break that guarantee, and would do so precisely in the rare situations where it matters most.

6. How a producer proves it controls the stake

It does not prove it separately. The chain checks two things:

1. The staking output exists and is unspent, so its weight is in the registry.
2. The block's elected leader key *is* the public key embedded in that staking script.

The proof of control is the block signature itself. The per-block challenge is `<leader> OP_CHECKSIG`, so only the holder of the corresponding private key can produce a valid block. The chain never asks anyone to sign over their coins and never inspects a producer's inputs.

One corollary deserves emphasis, because sections 8 and 9 exist to address it. In the plain arrangement, **the key that signs blocks is also the key that can spend the staked coins.** A block-signing key must be online continuously. Left unaddressed, that would force a staker to keep the key controlling its money on an internet-facing machine.

7. How a producer is paid

There is no subsidy, so a producer's reward is exactly the fees in its block. Consensus requires that *every* coinbase output carrying value pays the required payee, which by default is a pay-to-witness-public-key-hash of the elected leader's key. Zero-value outputs, such as commitments, are exempt. A block whose coinbase pays anyone else is invalid.

Two details matter downstream. Rewards land in a **separate output**, never inside the staking output,

so staking income is immediately spendable even when the stake itself is frozen. And because the default rule is "pay the leader, and only the leader," a pool operator producing blocks with borrowed weight would collect the entire reward. Section 10 is the answer to that.

8. Staking-only periods: tokens that earn but cannot be sold

Sequentia's supply is distributed at genesis, with allocations subject to a cliff, a linear vesting schedule, and a **staking-only period**. Tokens unlock for staking first, and become liquid, meaning sellable or transferable to third parties, only some months afterwards. An allocation might, for instance, unlock a slice for staking in month seven that does not become liquid until month thirty-one.

The mechanism is coherent only because of the fact established in section 2: a stake never moves. A coin that literally cannot be spent still counts toward weight, still produces blocks, and still earns fees. Non-transferability is a direct consequence of non-spendability, so no covenant, no escrow, and no trusted custodian is involved.

Accordingly the staking script admits an optional **absolute** timelock. The output stakes throughout, and cannot be sold, transferred, or otherwise moved before that time. The weight calculation ignores spendability entirely, so it required no special case.

A relative timelock could not serve this purpose. Relative locks are sixteen bits wide, so the time-based encoding tops out at about 388 days and the height-based encoding, at thirty-second slots, at under 23 days. A multi-year staking-only period is simply not expressible as a relative lock. Absolute locks are thirty-two bits and reach any calendar date. Because block heights drift against a calendar over a multi-year schedule, absolute locks in these outputs should be expressed as wall-clock times rather than heights.

Enforcing the cliff, as distinct from the liquidity date, is a matter of not making the output a staking output yet: the tranche waits in an ordinary timelocked output and is converted into a staking output when the cliff ends. Where the transition itself must be trustless rather than merely observable, the pre-cliff output can be a tapscript covenant that permits exactly one destination, namely the staking script it will become. Every field of that destination is a constant known when the tranche is created, so this is a fixed hash comparison rather than a recursive covenant.

9. Delegation: lending the right to produce, never the coins

Section 6 ended on a hazard: one key both signs blocks and spends the coins. Section 8 sharpens it, because a staking output frozen for years cannot be respent, so a producer named *inside* such an output could never be replaced. A compromised block-signing key would be irrevocable for the whole lock.

The resolution is an indirection. A staker, called the **controller**, lends its weight to a **signer** by funding a separate, small record:

```
<"SEQDEL"> OP_DROP <signer> OP_DROP <controller> OP_CHECKSIG
```

While that record is unspent, the controller's weight counts toward the signer, and the signer is the key that must produce and sign blocks. The record is spendable by the **controller alone**. The staking script is untouched; its public key is now understood as the cold key that controls the coins.

Three properties follow at once.

- **Custody is never transferred.** The signer does not appear in the staking script's `OP_CHECKSIG` and therefore can never spend the stake. This is not a promise or a contract; the signer does not hold the key. The coins do not move.
- **The hot key problem dissolves.** A staker delegates to its own online key and keeps the coin-controlling key offline. If the block-producing server is compromised, the attacker can sign blocks and nothing else.
- **Rights can be rotated inside a vesting lock.** Re-pointing the record, or reclaiming it, spends only the record and never the staking output. It works while the stake is frozen for years.

For an ordinary holder, this is what a staking pool is on Sequentia. A pool operator's key is named in the record; the operator produces blocks using the holder's weight; the holder's coins never leave the holder's own output, spendable by nobody else. Leaving is unilateral and immediate: spend the record. No operator consent, no unbonding queue, no movement of funds. Holders below the minimum-stake floor, who cannot stake alone at all, can participate this way.

Delegation resolves in **one hop and is never chained**. If A delegates to B and B delegates to C, then A's weight counts for B while B's own weight counts for C. Following chains would admit cycles.

Two rules keep the mechanism sound. A rotation spends the old record and creates the new one in a single transaction, so the removal of the old record must not erase the new one; removal is conditional on the record still being the one in force, and the disconnect path is its exact inverse. And consensus forbids two unspent records for one controller, because two would make the signer, and therefore the leader election, ambiguous and node-dependent, which is to say a chain split.

10. Payout policies: how a pool's rewards are distributed

Delegation protects the coins but says nothing about the rewards. Under the default rule of section 7, an operator producing blocks with borrowed weight collects all of the fees and is trusted to pass a share back. A producer may instead **commit** to a payout policy, announced on-chain in a record spendable by itself:

```
<"SEQPAY"> OP_DROP <activation> OP_DROP <mode> OP_DROP <parameter> OP_DROP <signer>
OP_CHECKSIG
```

Two modes exist, and operators and delegators choose between them.

Direct mode

The coinbase must pay a committed script. The operator cannot silently redirect the reward. It is important to be exact about what this does and does not guarantee: **the chain does not check that**

the destination is fair to anyone. An operator may commit to its own address and keep everything. Direct mode is a public, binding commitment about where rewards go, not a proof that they are shared. It is trust-minimising, not trustless.

Lottery mode

The coinbase must pay **one participant**, drawn from everyone whose weight counts toward that signer, weighted by that weight. The draw is derived from the block's election seed, which comes from Bitcoin's proof of work, so no operator can bias it. Every node computes the same winner, and a block paying anyone else is invalid.

Over many blocks each delegator earns its exact proportional share, enforced by consensus, with no per-delegator accounting and no epoch reward subsystem. The operator retains a commission, expressed in basis points and visible to everyone. At zero commission the operator still earns on its own stake, as a participant like any other, but takes no share of what its delegators earn.

The cost is real and should not be glossed. **Lottery mode does not smooth income.** A delegator holding one percent of a pool does not receive one percent of every block; it receives the whole of one block in a hundred. The expected value is exactly right and the variance is not reduced. Since smoothing income is one of the two reasons pools exist, lottery mode delivers trustlessness at the price of that benefit. The other reason survives untouched: holders below the minimum-stake floor can participate at all.

The notice period

A policy cannot bind until its stated activation height, which consensus requires to be at least the notice period, about a day, beyond the block announcing it. Announcing does not cancel an earlier policy. The policy in force at a given height is the announced policy with the greatest activation at or below it, so a pending policy and the one it will replace coexist unambiguously throughout the notice window.

The reasoning deserves to be spelled out, because without it the design would be decorative. A delegator can leave a pool **instantly and unilaterally**, without the operator's consent, without waiting for an unbonding period, and without the coins moving. Inspecting a pool before delegating is therefore meaningful only if what was inspected cannot change silently afterwards. With no notice period, an operator could accept delegations, redirect the rewards on the very next block, and collect. With one, a hostile change sits visible on-chain for a day before it pays a single coin, and any delegator can exit first.

Between them, exit and notice do the work that a fairness rule cannot. The chain guarantees that a commitment is public, that it cannot change without warning, and that nobody is ever trapped.

11. The trust model, stated plainly

It is worth separating what mathematics guarantees from what incentives and vigilance guarantee.

Guaranteed by the protocol. History buried beneath a Bitcoin checkpoint cannot be rewritten without rewriting Bitcoin's proof of work, so stakers who have unbonded and departed cannot forge an alternative past. A pool can never spend a delegator's coins. A delegator can always leave, immediately, without permission. A staking output under a vesting lock cannot be sold or transferred before its date, whatever its owner intends. A committed payout policy cannot change without a day's public notice. Under lottery mode, each participant's expected share of rewards is exact, and the draw cannot be biased by any participant, because its randomness is Bitcoin's.

Not guaranteed by the protocol. Direct mode does not ensure an operator's committed destination shares anything with its delegators; it only ensures the destination is public and stable. A notice period protects a delegator who *notices*, and no consensus rule can make anyone look, so wallets and explorers are expected to watch delegated pools and raise a warning when a change is announced. Lottery mode gives correct expected income but lumpy realisation; a delegator wanting smooth income must either accept a direct-mode operator's terms or hold enough stake to produce blocks itself. A pool operator that has never staked has nowhere to place its committee registration, since that key rides inside a staking output, and so must hold a stake of its own to join the committee.

What the design refuses to do is pretend. It does not claim that a pool is honest because it says so, and it does not build an accounting subsystem to enforce fairness that would then have to be trusted in its own right. It gives the delegator three things instead: custody that is never surrendered, an exit that is never blocked, and a commitment that cannot change behind their back. A delegator who wants no trust at all can choose a lottery pool and rely on nothing but Bitcoin's dice.

12. Reference

Consensus rejections specific to staking

condition	meaning
coinbase does not pay the required payee	the block's fees go somewhere other than the leader, or the payout policy in force
two delegation records for one controller	the signer, and hence the election, would be ambiguous
a delegation record created while another is unspent	as above
a payout policy binding before its notice elapses	the announcement was not given the required warning
two payout records at one activation height	the policy in force at that height would be ambiguous
a staking output whose BLS	one committee key per staker

condition	meaning
registration conflicts	

Node interfaces

call	purpose
`getstakescript`, `registerstake`	build and fund a staking output, optionally with a committee registration and a vesting locktime
`getstakerinfo`	live stake weights, by signer
`getdelegationscript`, `getdelegationinfo`	lend or reclaim block-signing rights; read the live controller-to-signer map
`getpayoutscript`, `getpayoutinfo`	announce a payout policy; inspect what each producer has committed to and whether it is yet in force

13. Summary

A stake on Sequentia is a coin sitting still in a recognisable output. Bitcoin's proof of work supplies the randomness that elects who produces the next block, weighted by how much is sitting there, and a committee of the top-ranked stakers certifies it with a single aggregated signature. A certified block is final at once; a block whose reference has been buried in Bitcoin for a fortnight is final beyond the reach of anyone who has since sold their stake; and a block whose Bitcoin anchor is orphaned is discarded regardless, because no rule of Sequentia's may outrank the chain it is anchored to. Because a coin never moves in order to earn, a coin that is forbidden to move can still earn, which is what a staking-only vesting period is. Because the right to produce blocks is named in a separate small output rather than inside the coin, that right can be lent to a pool, reclaimed instantly, and rotated even while the coin is frozen for years, and the pool can never touch the coin. And because a block's reward must go where its producer publicly committed it a day in advance, a delegator can read what a pool promises, watch it, and leave before any change takes effect, or else choose a pool that has handed the decision to Bitcoin's dice and promised nothing at all.

Sequentia is developed by Concatena Labs. The network is called Sequentia; its token is called Sequence, ticker SEQ.